Hough Transform

https://liaoux.github.io/uir-cv-2026/

Introduction to Computer Vision
Spring 2026, Lecture 5

Hough Transform

We use the Hough Transform to detect geometric shapes from edge pixels.

Topics:

1. Line Detection
2. Circle Detection

Difficulties for the Fitting Approach

Original Image

Edge map

How do we detect shapes like circles in a complex, noisy edge map?

Why Is It So Difficult?

Original Image

Edge map

1. Extraneous Data — Which points to fit to?
2. Incomplete Data — Only part of the model is visible.
3. Noisy Edges — Edges may be miss-detected

What is the Solution?

Original Image

Edge map

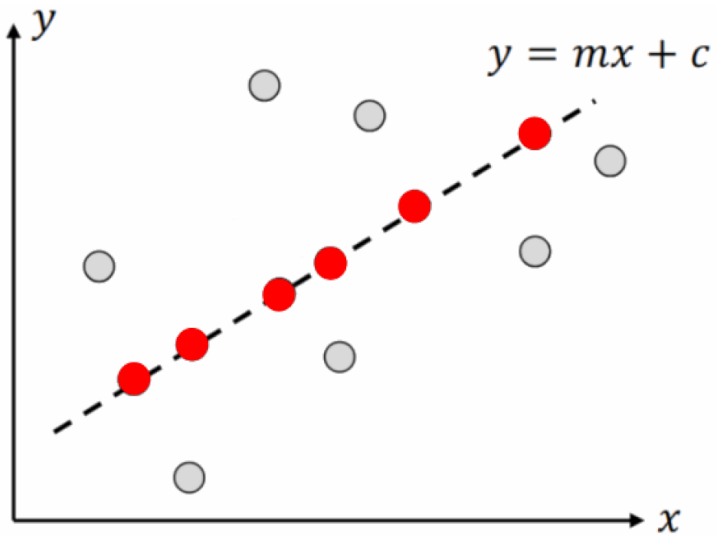
Solution: Hough Transform (1960s)

Hough Transform: Line Detection

Hough Transform: Line Detection

Given: Edge Points (x_i, y_i)

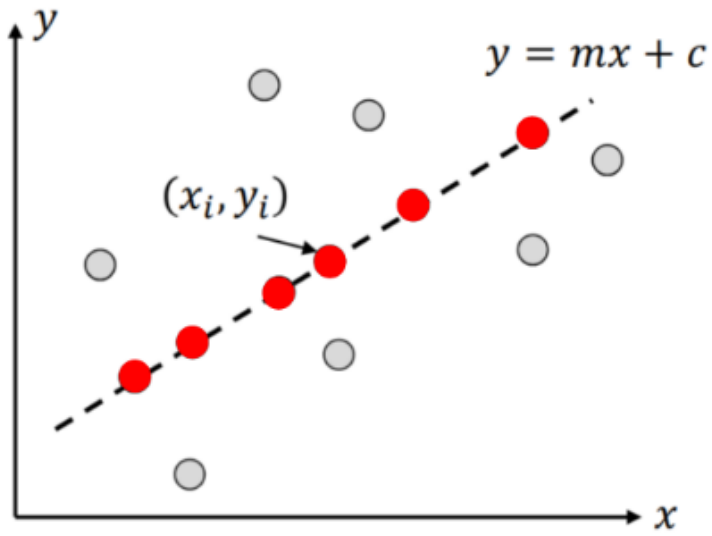
Task: Detect line: $y = mx + c$



Hough Transform: Line Detection

Given: Edge Points (x_i, y_i)

Task: Detect line: $y = mx + c$



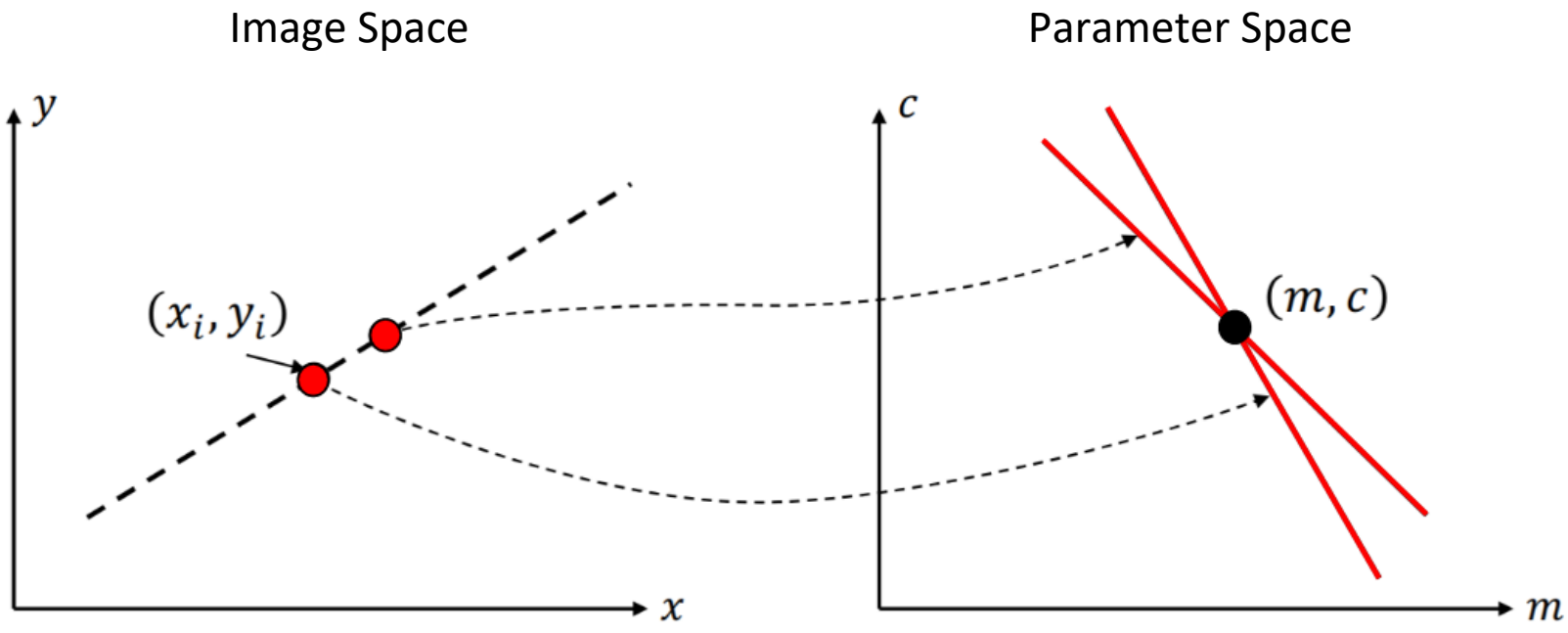
Consider a point on the straight line (x_i, y_i):

$$y_i = mx_i + c \quad \Longleftrightarrow \quad c = -mx_i + y_i$$

This is a straight line in x-y space

This is a straight line in m-c space

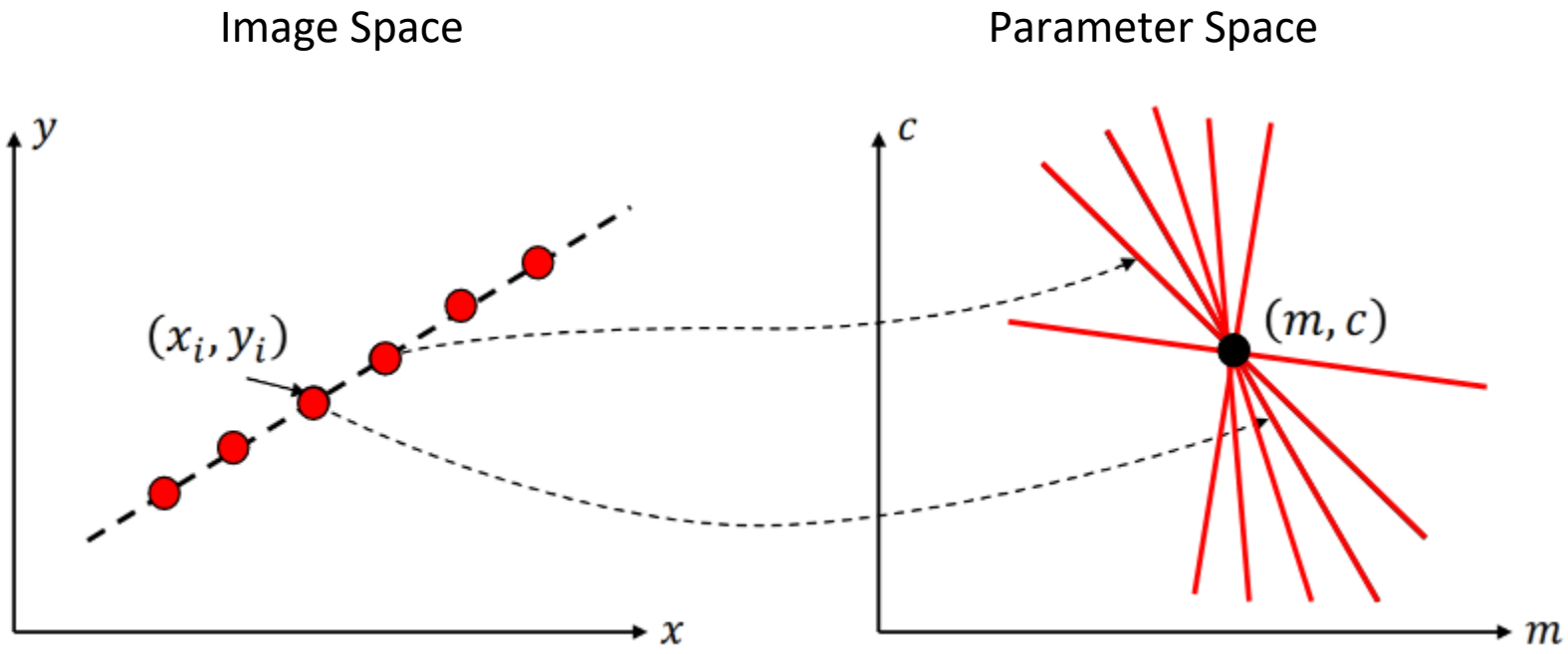
Hough Transform: Concept



$$y_i = mx_i + c \qquad\qquad c = -mx_i + y_i$$

Each red line in parameter space corresponds to all the lines that pass through **each** point (x_i, y_i) in image space.

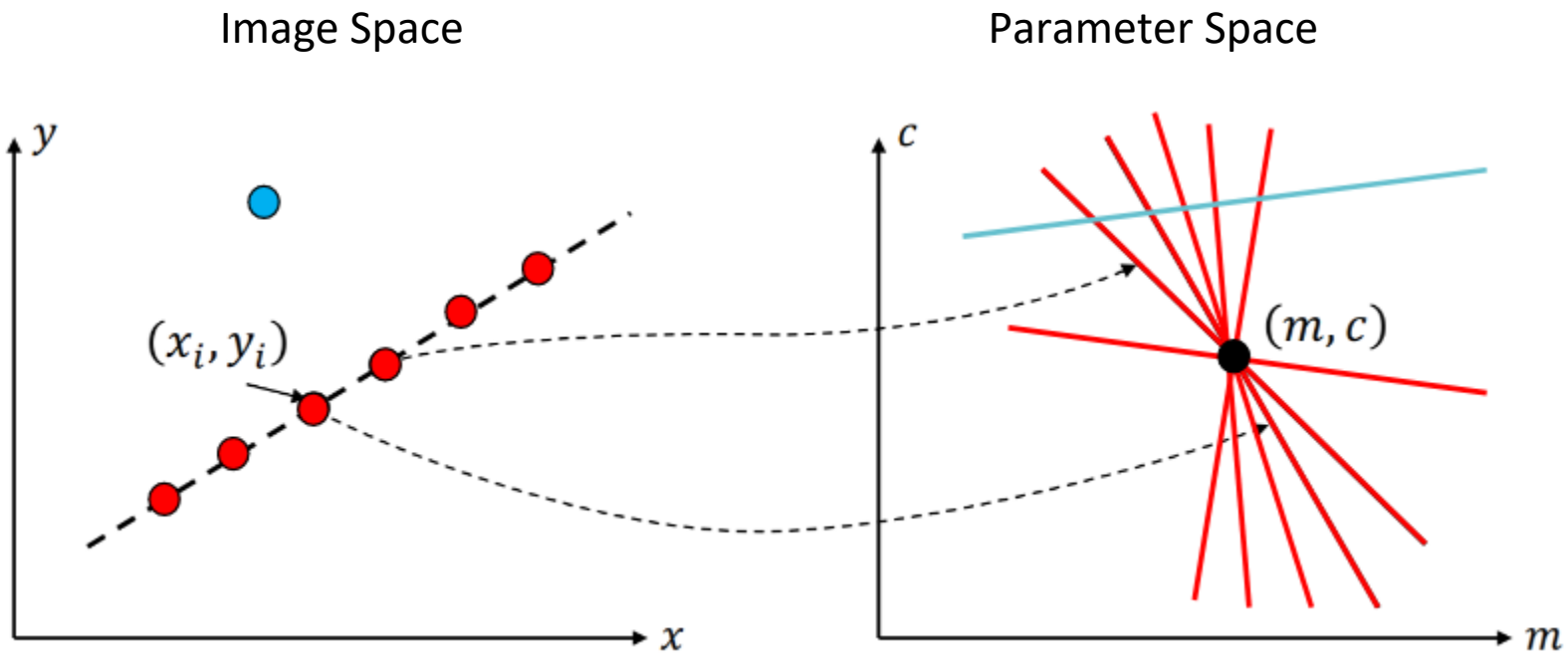
Hough Transform: Concept



$$y_i = mx_i + c \qquad\qquad c = -mx_i + y_i$$

All points on the same line in image space map to lines in parameter space that intersect at a single point (m, c) — the parameters of that line.

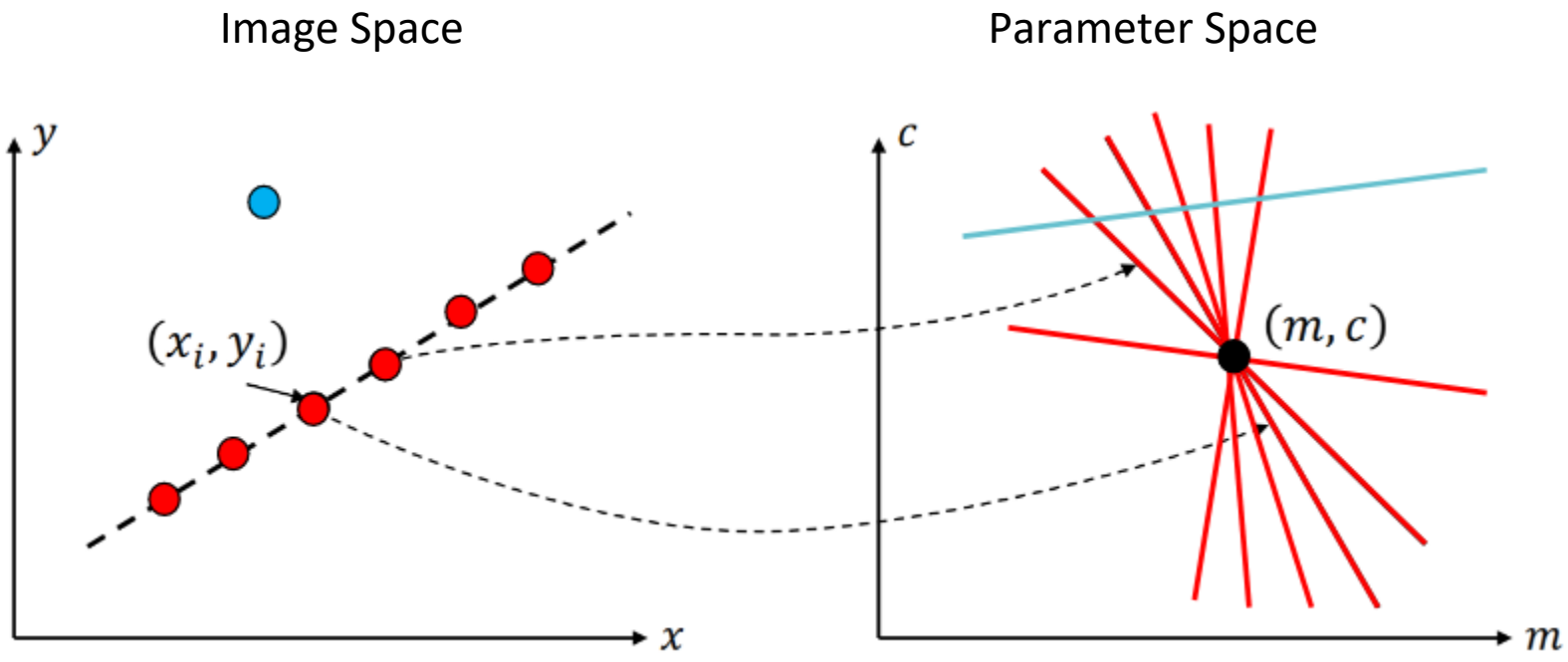
Hough Transform: Concept



$$y_i = mx_i + c \qquad\qquad c = -mx_i + y_i$$

A point **NOT** on the line in image space also maps to a line in parameter space — but it won't pass through (m, c).

Hough Transform: Concept



Point ⟷ Line

Line ⟷ Point

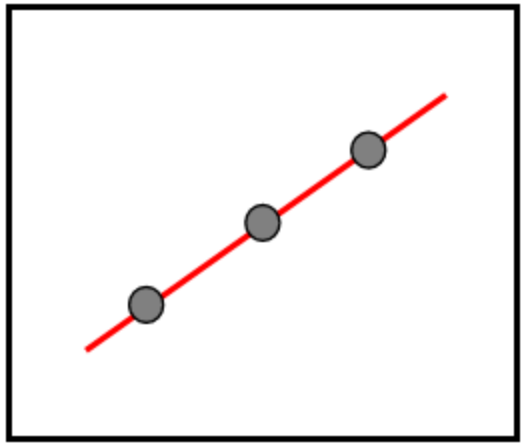
Line Detection Algorithm

Step 1. Quantize parameter space (m,c)

Step 2. Create **accumulator array** $A(m,c)$

Step 3. Set $A(m,c) = 0$ for all (m,c)

Image



$A(m,c)$

c	0	0	0	0	0
	0	0	0	0	0
	0	0	0	0	0
	0	0	0	0	0
	0	0	0	0	0
					m

Line Detection Algorithm

Step 1. Quantize parameter space (m,c)

Step 2. Create **accumulator array** $A(m,c)$

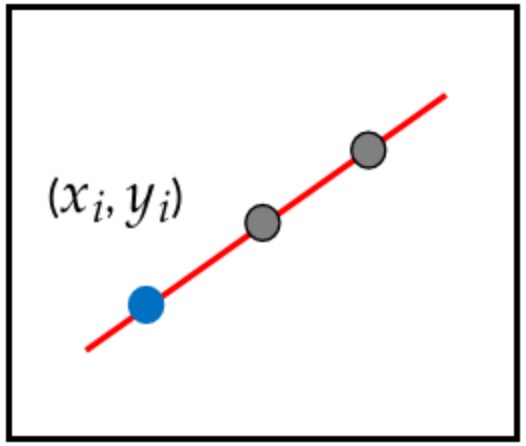
Step 3. Set $A(m,c) = 0$ for all (m,c)

Step 4. For each edge point (x_i, y_i),

$$A(m,c) = A(m,c) + 1$$

if (m,c) lies on the line: $c = -mx_i + y_i$

Image



$A(m,c)$

c	1	0	0	0	0	
	0	1	0	0	0	
	0	0	1	0	0	
	0	0	0	1	0	
	0	0	0	0	1	m

Line Detection Algorithm

Step 1. Quantize parameter space (m,c)

Step 2. Create **accumulator array** $A(m,c)$

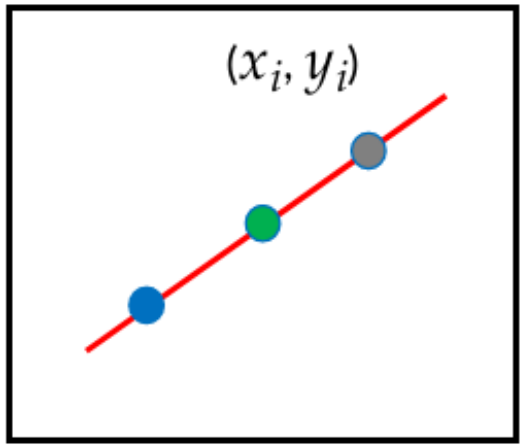
Step 3. Set $A(m,c) = 0$ for all (m,c)

Step 4. For each edge point (x_i, y_i),

$$A(m,c) = A(m,c) + 1$$

if (m,c) lies on the line: $c = -mx_i + y_i$

Image



$A(m,c)$

c	1	0	0	0	1	
	0	1	0	1	0	
	0	0	2	0	0	
	0	1	0	1	0	
	1	0	0	0	1	m

Line Detection Algorithm

Step 1. Quantize parameter space (m,c)

Step 2. Create **accumulator array** $A(m,c)$

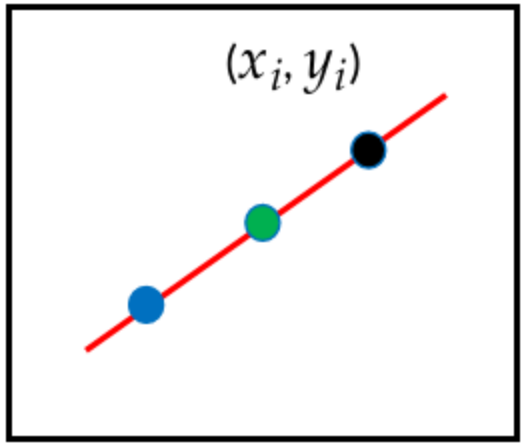
Step 3. Set $A(m,c) = 0$ for all (m,c)

Step 4. For each edge point (x_i, y_i),

$$A(m,c) = A(m,c) + 1$$

if (m,c) lies on the line: $c = -mx_i + y_i$

Image



$A(m,c)$

c	1	0	0	0	1	
	0	1	0	1	0	
	1	1	3	1	1	
	0	1	0	1	0	
	1	0	0	0	1	m

Line Detection Algorithm

Step 1. Quantize parameter space (m,c)

Step 2. Create **accumulator array** $A(m,c)$

Step 3. Set $A(m,c) = 0$ for all (m,c)

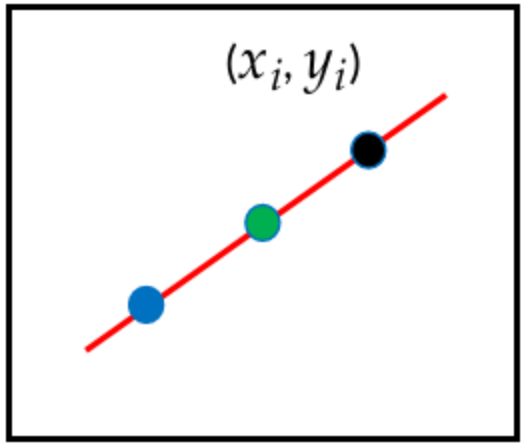
Step 4. For each edge point (x_i, y_i),

$$A(m,c) = A(m,c) + 1$$

if (m,c) lies on the line: $c = -mx_i + y_i$

Step 5. Find local maxima in $A(m,c)$

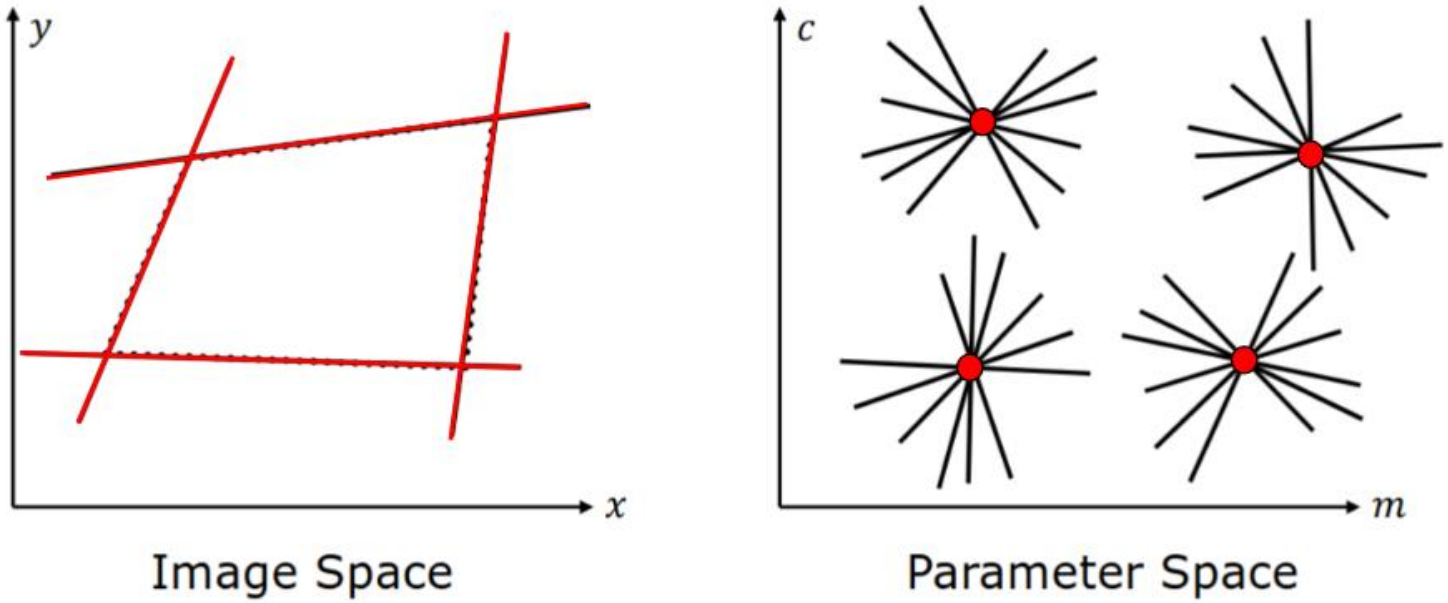
Image



$A(m,c)$

c	1	0	0	0	1	
	0	1	0	1	0	
	1	1	3	1	1	
	0	1	0	1	0	
	1	0	0	0	1	m

Multiple Line Detection



The **Hough Transform** detects **multiple lines** simultaneously — by repeating the **voting process** for each edge point, strong **maxima** in parameter space reveal lines without needing to know which point belongs to which line.

Better Parameterization

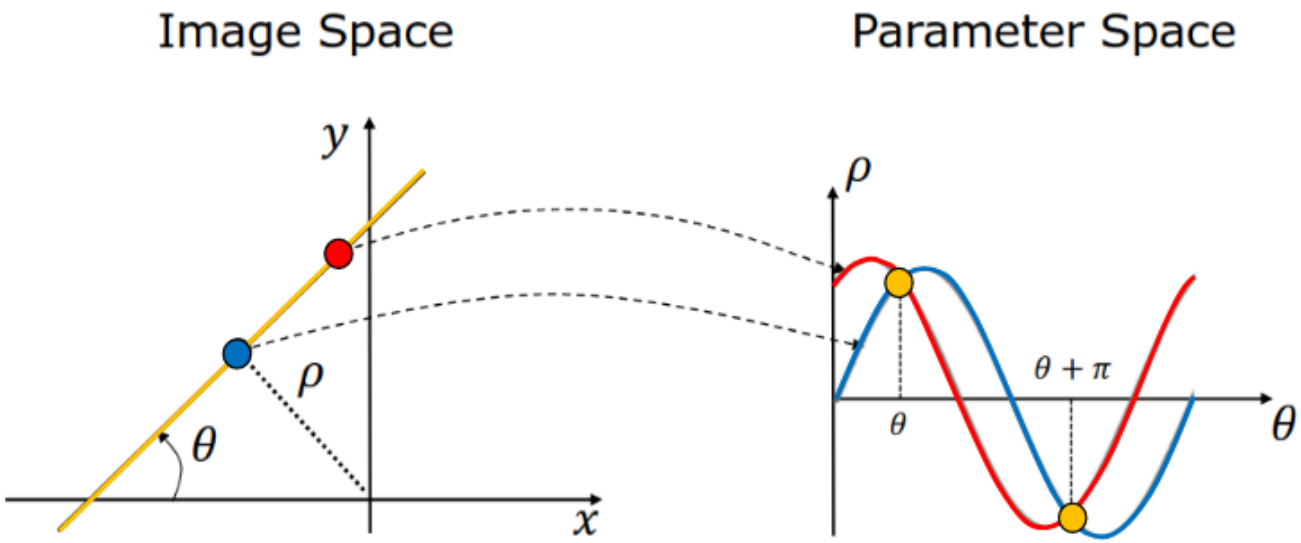
Issue: Using $y_i = mx_i + c$, slope m ranges from $-\infty$ to $+\infty$

- Requires a massive accumulator array
- Costly in memory and computation

Solution: Use the equation $x sin\theta - y cos\theta + \rho = 0$

- θ (orientation) is finite: $0 \leq \theta < \pi$
- ρ (distance from origin) is finite: bounded by image size

Better Parameterization



$x\sin\theta + y\cos\theta + \rho = 0$ $\qquad$ $x\sin\theta + y\cos\theta + \rho = 0$

In this case, the Hough transform uses an accumulator array with the parameters ρ and θ, and **voting** is done along sinusoids.

Hough Transform: Practical Considerations

Practical Consideration 1: Accumulator Cell Size

- The accumulator array is divided into cells — but how big should they be?
- **Too large** → similar lines merge into one cell → false detections
- **Too small** → noise scatters votes across cells → real lines get missed
- *Cell size must balance resolution and noise tolerance*

Hough Transform: Practical Considerations

Practical Consideration 2: Finding the Maxima

- After voting, we need to find the peaks in the accumulator array
- In practice, peaks are **blurred** due to:
  - Image noise
  - Finite resolution of the array

- Solution: Use a peak-finding algorithm
  - e.g., Non-maximal suppression (same method used for corner detection)

Hough Transform: Practical Considerations

Practical Consideration 3: Inaccurate Edge Locations

- Lines in real images are **not perfectly straight**
- Voting for a **single cell** at a time is too strict and fragile

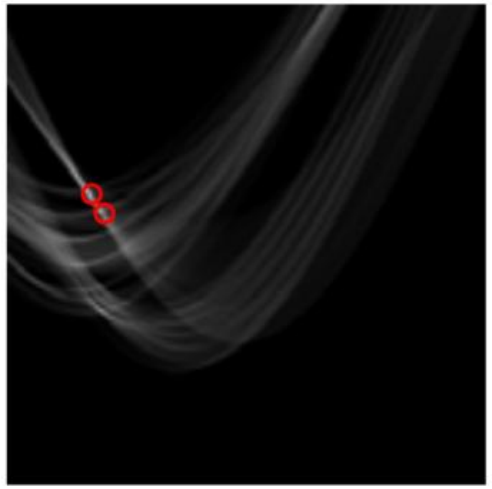
- Better approach: **Vote for a small patch of cells**
  - Strongest vote at the **center** of the patch
  - Vote strength **tapers off** toward the edges

Line Detection Results

Original Image

Gradient

Edge (Threshold)

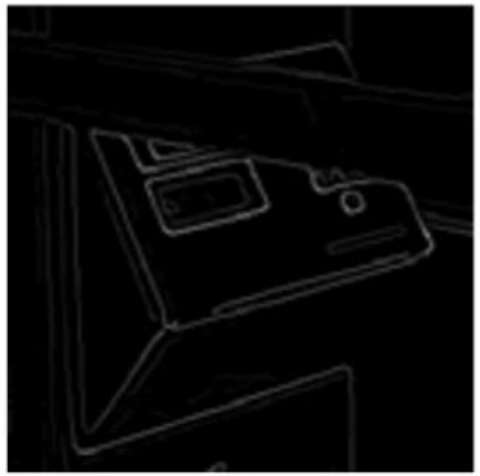


Hough Transform $A(\rho, \theta)$

Detected Lines

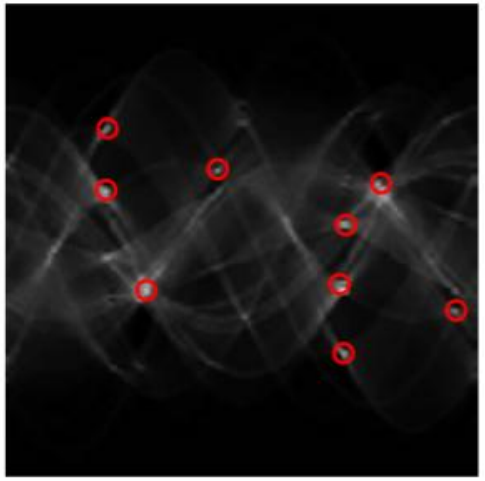
Line Detection Results

Original Image

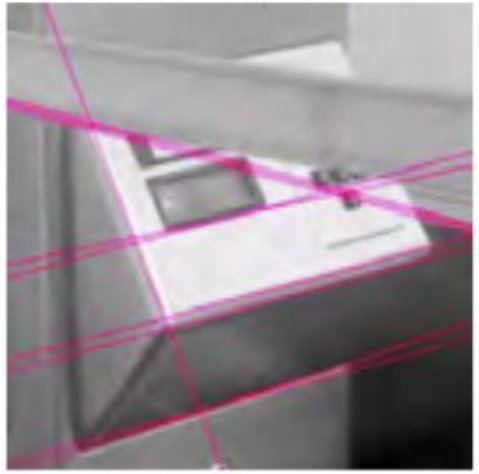


Gradient

Edge (Threshold)



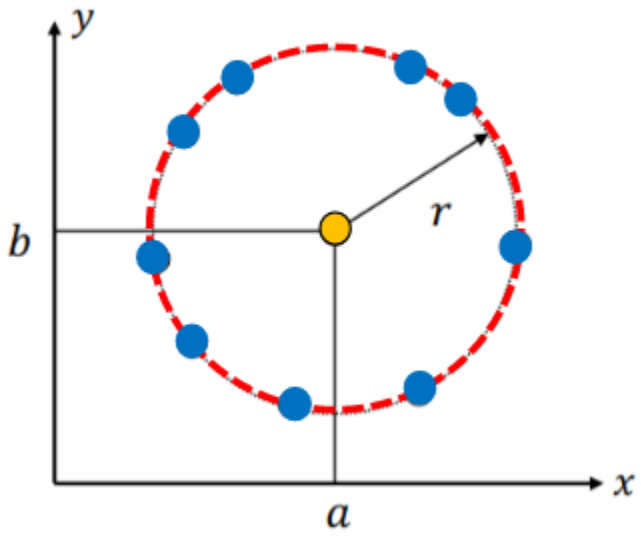
Hough Transform $A(\rho, \theta)$



Detected Lines

Hough Transform: Circle Detection

Hough Transform: Circle Detection

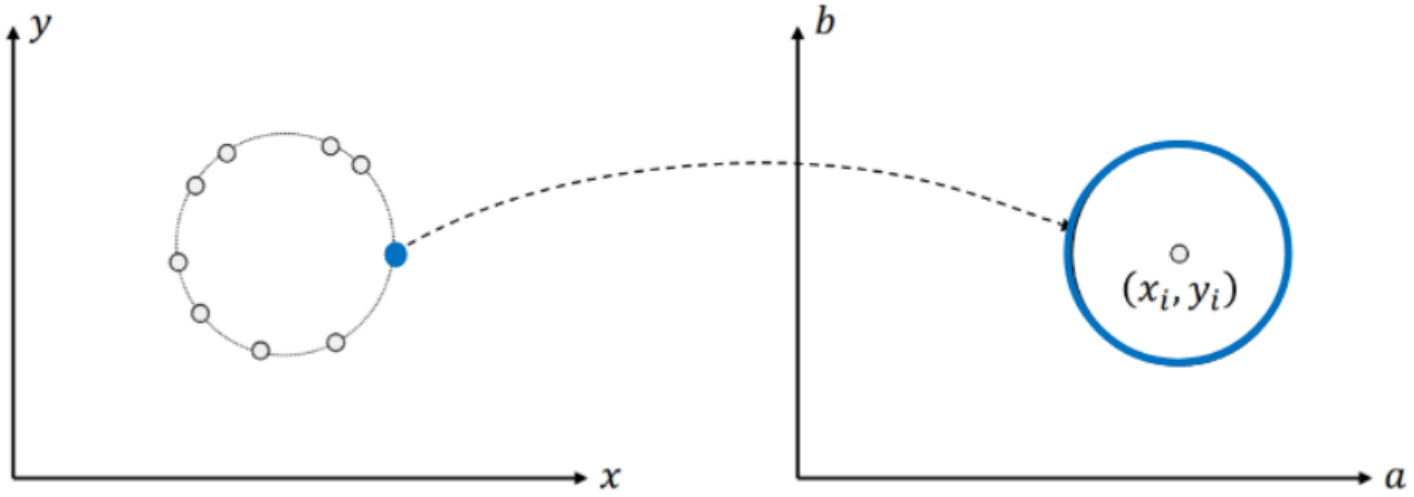


Equation of Circle: $(x_i - a)^2 + (y_i - b)^2 = r^2$

Hough Transform: Circle Detection

If radius r is known: Accumulator Array: $A(a,b)$

Image Space | Parameter Space



$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

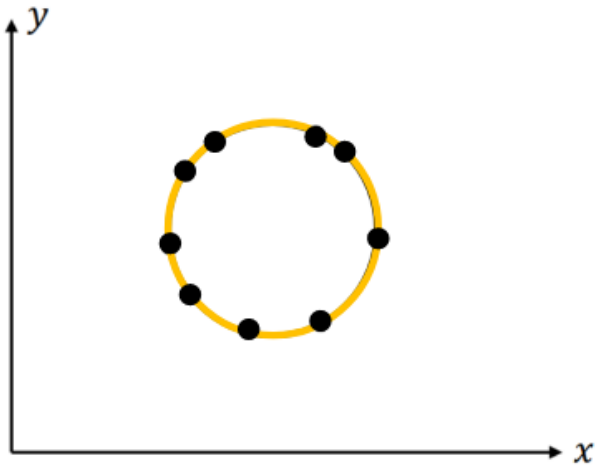
$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

Each point in the image will map to **a** circle in parameter space

Hough Transform: Circle Detection

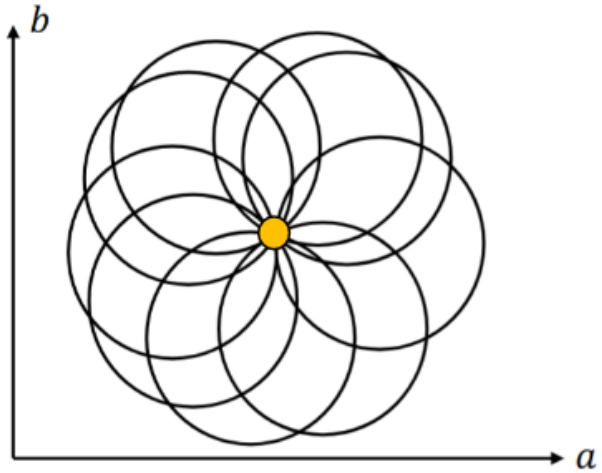
If radius r is known: Accumulator Array: $A(a,b)$

Image Space



$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

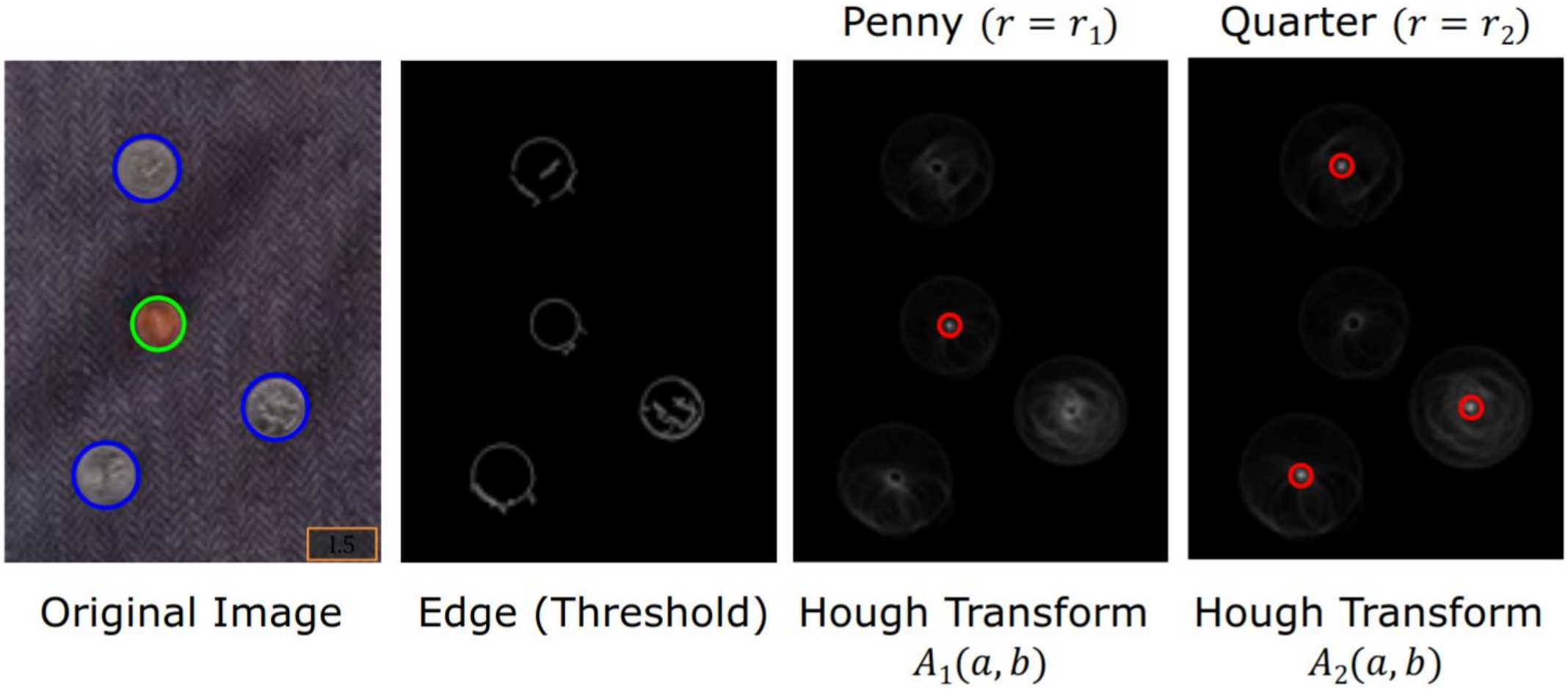
Parameter Space



$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

Each point in the image will map to **a** circle in parameter space

Circle Detection Results

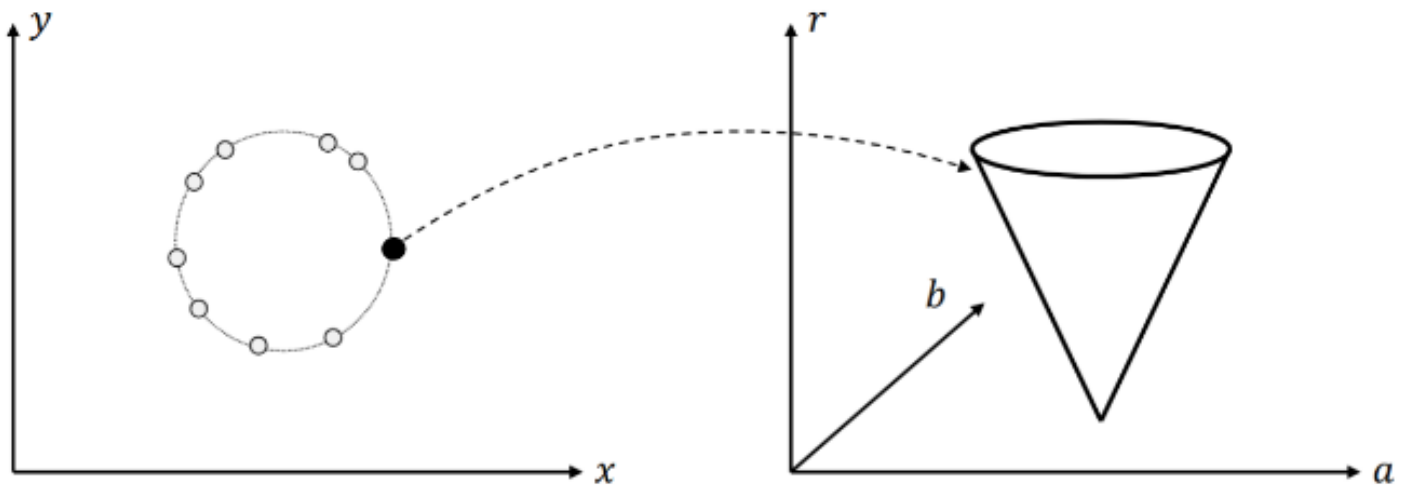


Hough Transform: Circle Detection

If radius r is unknown: Accumulator Array: $A(a,b,r)$

Image Space

Parameter Space



$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

$$(x_i - a)^2 + (y_i - b)^2 = r^2$$

Each edge point (x_i, y_i) **votes** along a **cone** in parameter space

Hough Transform: Limitations

- **Computational cost** grows exponentially with the number of unknown parameters
- **Memory intensive** — accumulator array size increases rapidly in higher dimensions
- **Sensitive to cell size** — too large or too small leads to wrong detections
- **Works best for simple shapes** — lines, circles, ellipses; struggles with complex shapes (Using the Generalized Hough Model).
- **Not practical for shapes with many parameters**

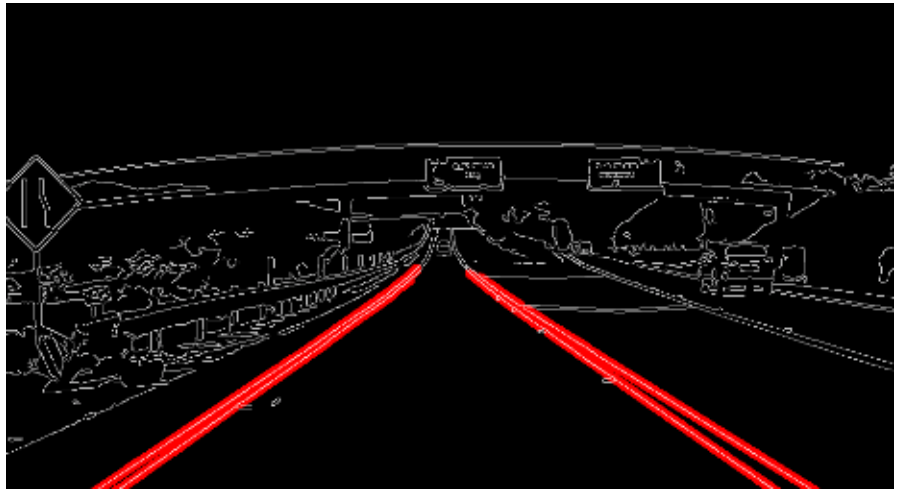
Hough Transform: Applications

Biometrics & Human Interaction — The Hough Transform can detect and localize features for face and fingerprint recognition, as well as track body parts for gesture recognition in human-machine interaction.

Taneja, N., Shabaz, M., & Khajuria, V. (2018). Iris detection using segmentation techniques. International journal of computer sciences and engineering, 6(9), 442-444.

Hough Transform: Applications

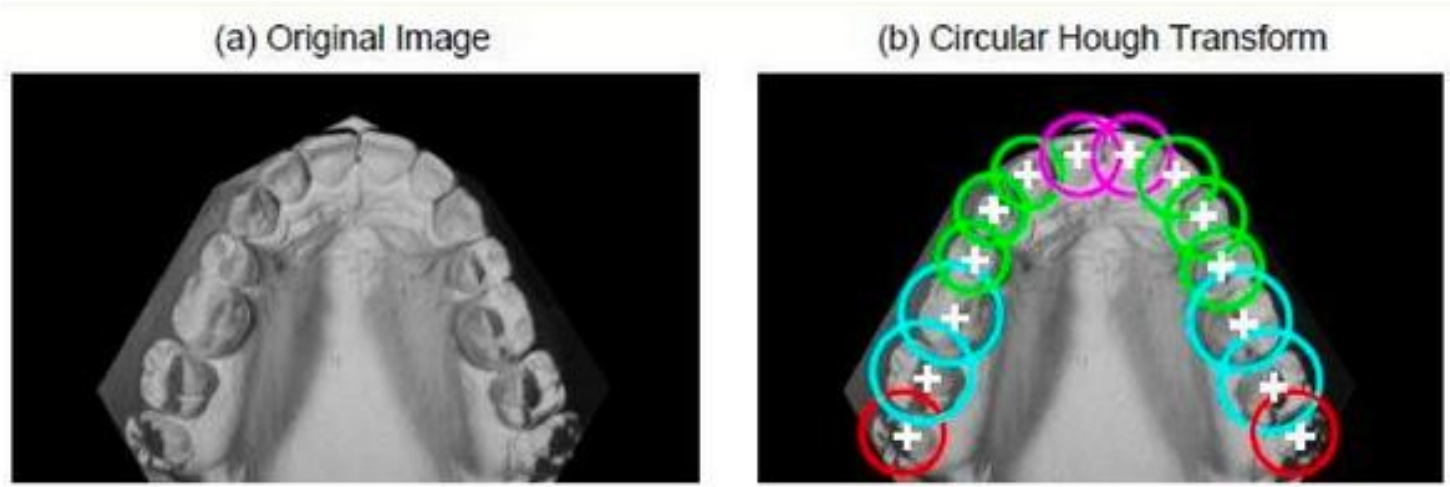
Lane Detection — In autonomous driving, the Hough Transform detects straight and curved road lane markings from edge-detected camera frames, enabling the vehicle to stay within its lane and navigate safely in real time.



https://hortovanyi.wordpress.com/tag/hough-transform/

Hough Transform: Applications

Medical Application — The Hough transform can be used in medical applications, such as image analysis or diagnosis, where the Hough transform can detect specific geometric shapes in medical images, such as tumors or blood vessels.



Yadollahi, M., Procházka, A., Vyšata, O., & Kašparová, M.
THE ORTHODONTIC PLASTER CAST IMAGE SEGMENTATION.